Graphs and Algorithms
Lecture Notes for COMP40008
2025-2026

Iain Phillips

Contents

1 Graphs and Graph Algorithms **5**

1.1 Graphs—Basics 5

1.1.1 Definitions 5

1.1.2 Representations of Graphs 6

1.1.3 Big-Oh Notation 8

1.2 Graph Isomorphism and Planar Graphs 8

1.2.1 Isomorphism 8

1.2.2 Planar Graphs 10

1.2.3 Colouring Maps and Graphs 12

1.3 Connectedness, Euler Paths and Hamiltonian Circuits 13

1.3.1 Paths and Cycles 13

1.3.2 Euler Paths and Circuits 14

1.3.3 Hamiltonian Circuits 16

1.4 Trees 18

1.5 Directed Graphs 19

1.6 Graph Traversal 20

1.6.1 Depth-first and Breadth-First Search 20

1.6.2 Applications 22

1.7 Weighted Graphs 28

1.7.1 Minimum Spanning Trees 28

1.7.2 Prim's Algorithm 29

1.7.3 Kruskal's Algorithm 34

1.7.4 The Shortest Path Problem 40
1.7.5 The Travelling Salesman Problem 53

2 Algorithm Analysis **56**
2.1 Introduction 56
2.2 Searching an unordered list 57
2.3 Searching an ordered list 59
2.4 Orders of complexity 62
2.5 Strassen's Algorithm 64
2.6 Sorting 65
2.6.1 Lower Bound for Sorting by Comparison 66
2.6.2 Mergesort 69
2.6.3 Master Theorem 71
2.6.4 Quicksort 75
2.6.5 Heapsort 78
2.7 Dynamic Programming 85

3 Introduction to Complexity **88**
3.1 Tractable problems and P 88
3.2 The complexity class NP 91
3.3 Problem reduction 93
3.4 NP-completeness 94
3.4.1 Intractability via NP-completeness 95

Introduction

This course falls into three parts. In Part 1 we study graphs and graph algorithms. In Part 2 we analyse algorithms for searching and sorting. Some of the algorithms discussed in Part 2 have already been introduced and verified in 141 Reasoning about Programs. In Part 3 we give an introduction to Complexity Theory, and discuss NP-complete problems and the $\mathsf{P} = \mathsf{NP}$ question.

Acknowledgements. Thanks to Istvan Maros for pointing out some errors in an earlier version. I have benefited from consulting Ian Hodkinson's lecture notes on algorithms.

Reading

Main recommendations:

- T. Roughgarden, Algorithms Illuminated: Omnibus Edition. Cambridge 2022. Chapters 1-5, 7-10, 13, 15, 16, 18, 21.1. Note that Chapters 19-24 use a different definition of the complexity class NP, and a different notion of reduction, from the definitions we adopt in these lectures.
- S. Baase & A. Van Gelder, Computer Algorithms: Introduction to Design and Analysis. Third Edition. Addison-Wesley 2000. Chapters 1, 3.6-3.7, 4.1-4.9, 7-10, 13.
- Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C. (2022) Introduction to algorithms. 4th ed. Cambridge, Mass. ; London, MIT press. Chapters 1-3, 4.1-4.5, 6, 7, 8.1, 14.1-14.3, 19.1-19.3, 20.1-20.4, 21, 22.3, 23.2, 34.
- J. Gersting, Mathematical Structures for Computer Science. Seventh edition. Freeman, 2014. Chapters 2, 5, 6.

Also:

- S. Dasgupta, C.H. Papadimitriou, U. Vazirani, Algorithms. McGraw-Hill, 2008.
- Sedgewick, R., Wayne, K.D. (2011) Algorithms. 4th ed. Boston, Mass. ; London, Addison-Wesley.
- Cormen, T.H. (2013) Algorithms Unlocked. MIT Press.
- L. Fortnow, The Golden Ticket: P, NP, and the Search for the Impossible. Princeton University Press, 2013.

Synopsis

Part 1: Graphs, graph isomorphism, planar graphs, Euler paths, Hamiltonian circuits, trees, depth-first search, breadth-first search, weighted graphs, minimum spanning trees, Prim's algorithm, Kruskal's algorithm, Dijkstra's shortest path algorithm, travelling salesman problem, Floyd-Warshall algorithm.

Part 2: Time complexity, worst-case and average case analysis, decision trees, optimality, orders of complexity (big-Oh and Θ), Strassen's algorithm, master theorem for recurrence relations, binary search, quicksort, mergesort, heapsort. Dynamic programming

Part 3: Tractability, polynomial time, decision problems, the classes P and NP, many-one reduction, NP-completeness, the $\mathsf{P} = \mathsf{NP}$ problem

Chapter 1

Graphs and Graph Algorithms

1.1 Graphs—Basics

1.1.1 Definitions

Graphs are used to model networks of all kinds. They give rise to many interesting computational problems. It is easiest to think of graphs in terms of pictures. A graph is a set of points joined by lines. For instance a road map may be thought of as a graph, with the points being the towns and the lines the roads. In graph terminology, points are referred to as *nodes* (or *vertices*), and lines are referred to as *arcs* (or *edges*).

Figure 1.1 shows a graph with 7 nodes and 8 arcs. We have labelled the nodes. Notice that each arc has two endpoints. We see that graphs may have the following:

- *parallel (multiple) arcs*—two arcs with same endpoints (here 1 and 7)
- *loops*—a loop is an arc with both endpoints the same (here there is a loop on 3)

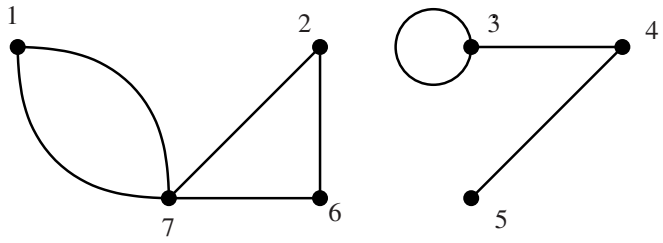


Figure 1.1: A graph

- *disconnected components*—here 1,2,6,7 and 3,4,5

Definition 1.1.1 *An* (undirected) graph *is a set N of nodes together with a set A of arcs, such that each $a \in A$ is associated with an unordered pair of nodes (the* endpoints *of a). A graph is* simple *if it has no parallel arcs and no loops.*

We will refer to graphs by $G, \ldots$. We talk about nodes(G) and arcs(G) in an obvious way. In these notes we only consider finite graphs.

Terminology 1.1.2 *An arc is said to be* incident *on its endpoints. A node n is said to be* incident *on any arc a which has n as one of its endpoints. An arc* joins *two nodes if they are its endpoints. Two nodes n, n' are* adjacent *if they are joined by some arc. The* degree *of a node is the number of arcs incident on it, where loops are counted twice. A node is said to be* odd (even) *if its degree is odd (even).*

For instance in Figure 1.1, the degrees are as follows:

node	1	2	3	4	5	6	7
degree	2	2	3	2	1	2	4

The total of all the degrees is 16, which is twice the number of arcs (8). This is true of all graphs. The reason is that each arc contributes twice to the total, once for each endpoint.

Theorem 1.1.3 *In any graph, the total of the degrees of all the nodes is twice the number of arcs. Moreover the number of odd nodes is even.*

There is a natural notion of subgraph:

Definition 1.1.4 *Let G_1, G_2 be graphs. We say that G_1 is a* subgraph *of G_2 if* nodes$(G_1) \subseteq$ nodes(G_2) *and if* arcs$(G_1) \subseteq$ arcs(G_2).

For example, in Figure 1.2 we see that G_1 is a subgraph of G_2. Notice that nodes in the subgraph G_1 may have fewer connections than nodes in G_2.

Any set $X \subseteq$ nodes(G) *induces* a subgraph $G[X]$ with nodes$(G[X]) = X$ and $G[X]$ inheriting all arcs of G between nodes in X. A graph G' is a *full* (or *induced*) subgraph of G if $G' = G[X]$ for some $X \subseteq$ nodes(G).

If G' is a subgraph of G and nodes$(G') =$ nodes(G), we say that G' *spans* G (or is a spanning subgraph of G).

1.1.2 Representations of Graphs

It is often useful to represent a graph by its *adjacency matrix*. Suppose that a graph has k nodes $n_1, \ldots, n_k$. Define

$$adj(i, j) = \text{the number of arcs joining } n_i \text{ to } n_j$$

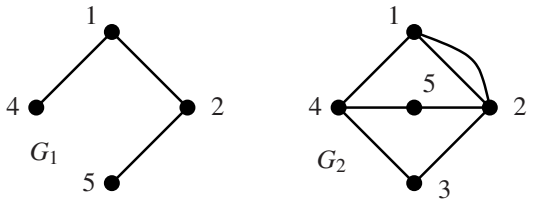


Figure 1.2: G_1 is a subgraph of G_2

For example, the graph in Figure 1.1 has a 7×7 adjacency matrix whose first two rows are:

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 \end{pmatrix}$$

As an exercise, complete the remaining 5 rows. You will notice that the matrix is symmetric. This is because the arcs are undirected, so that if node i is joined to j then j must be joined to i. In directed graphs (which we are not considering at present), the arcs have direction—one endpoint is the start and the other the finish. In diagrams the arcs will have arrows on them. We saw such diagrams when considering relations. If we had chosen a different order for the nodes, then we would have obtained a different matrix—the difference being that the rows and columns are reordered (in a consistent way). So a single graph can have a number of adjacency matrices, corresponding to different enumerations of the elements.

If a graph has n nodes then there are n^2 entries in the adjacency matrix. When dealing with a *sparse* graph, i.e. one with much fewer than n^2 arcs, then it can save time and space to represent the graph by its *adjacency list representation*, rather than by its adjacency matrix. The adjacency list representation consists of an array of n pointers, where the ith pointer points to a linked list adj[i] of the nodes which are adjacent to node i. For example, the graph in Figure 1.1 has the following adjacency list representation:

$$\begin{array}{lllllllll}
1 & \to & 7 & \to & 7 & & & & \\
2 & \to & 6 & \to & 7 & & & & \\
3 & \to & 3 & \to & 4 & & & & \\
4 & \to & 3 & \to & 5 & & & & \\
5 & \to & 4 & & & & & & \\
6 & \to & 2 & \to & 7 & & & & \\
7 & \to & 1 & \to & 1 & \to & 2 & \to & 6
\end{array}$$

Notice that each arc is recorded twice (except for loops, which only appear once). If there are n nodes and m arcs, then the total number of entries in the adjacency list representation is bounded by $n + 2m$.

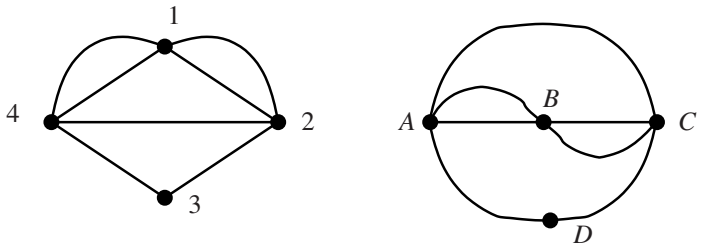


Figure 1.3: Two isomorphic graphs

1.1.3 Big-Oh Notation

Suppose that we wish to multiply together two $n \times n$ matrices, as in the following example for $n = 3$.

$$\begin{pmatrix} 1 & 0 & 2 \\ 3 & 1 & 0 \\ 4 & 2 & 3 \end{pmatrix} \begin{pmatrix} 3 & 6 & 1 \\ 2 & 0 & 5 \\ 1 & 4 & 2 \end{pmatrix} = \begin{pmatrix} 5 & 14 & 5 \\ 11 & 18 & 8 \\ 19 & 36 & 20 \end{pmatrix}$$

Each entry in the product takes n multiplications and $n-1$ additions. Since there are n^2 entries, the total number of multiplications is n^3, the total number of additions is $n^3 - n^2$, and the total number of arithmetical operations is $2n^3 - n^2$. Often we don't wish to be so precise. So we use the 'big-Oh" notation; $O(n^k)$ means bounded by some constant factor times n^k. Thus we can say that the total number of multiplications is $O(n^3)$, the total number of additions is $O(n^3)$, and the total number of arithmetical operations is $O(n^3)$.

For some values of k we use special names:

$O(1)$	constant
$O(n)$	linear (in n)
$O(n^2)$	quadratic (in n)

We shall give the formal definition of $O(n^k)$ later.

1.2 Graph Isomorphism and Planar Graphs

1.2.1 Isomorphism

Consider the two graphs in Figure 1.3. Although the nodes are labelled differently, they are connected in the same way. To make this similarity more precise, let us see how the individual nodes correspond. First, 3 matches with D, since they are the only nodes with degree two. Next, 1 matches with B, since they are not joined to 3, D respectively. Finally 2 matches with A, and 4 matches with C (we could equally well match 2 with C

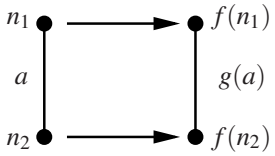


Figure 1.4: Isomorphism

and 4 with A). We have set up a bijection between the two sets of nodes. What is more, this is a bijection which preserves the connectedness of the nodes. Thus the adjacency matrix of the first graph is

$$\begin{pmatrix} 0 & 2 & 0 & 2 \\ 2 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 2 & 1 & 1 & 0 \end{pmatrix}$$

If we order the nodes of the second graph as B, A, D, C we get the same matrix. The bijection we have just given is an example of a graph isomorphism.

Definition 1.2.1 *Let G, G' be graphs. An* isomorphism *from G to G' is a bijection $f : \text{nodes}(G) \to \text{nodes}(G)'$ together with a bijection $g : \text{arcs}(G) \to \text{arcs}(G)'$ such that for any arc $a \in \text{arcs}(G)$, if the endpoints of a are $n_1, n_2 \in \text{nodes}(G)$, then the endpoints of $g(a)$ are $f(n_1)$ and $f(n_2)$. See Figure 1.4. If there is an isomorphism from G to G' we say that G is* isomorphic *to G', or that G and G' are isomorphic.*

It follows that if G is *isomorphic* to G' and $n_1, n_2 \in \text{nodes}(G)$, then $f(n_1)$ and $f(n_2)$ are connected by the same number of arcs in G' as n_1 and n_2 are connected by in G. If two graphs are isomorphic then they must clearly have the same number of nodes. Also their adjacency matrices must be the same, apart from the rows and columns possibly being reordered. If we wish to test whether two graphs are isomorphic, there are a number of obvious checks we can make first. For instance

- do they have the same number of nodes?
- do they have the same number of arcs?
- do they have the same number of loops?
- do their nodes have the same degrees (possibly reordered)?

If they fail tests of this kind then they can't be isomorphic. Suppose that they pass the initial tests. Then we attempt to set up the bijection on nodes. We must match nodes with the same degree, and preserve connectedness, as in the example at the start of the section. If we get the adjacency matrices to match then we know we have been successful. In practice with small graphs the eye can detect an isomorphism, or a reason why there can be no isomorphism, fairly easily. However an algorithm to check

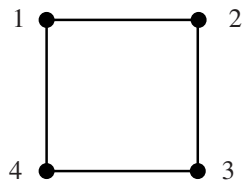


Figure 1.5: A square graph

whether two arbitrary graphs with n nodes each were isomorphic would appear to have to check the $n!$ possible different bijections. This makes the task apparently intractable.

Definition 1.2.2 *Let G be a graph. An* automorphism *on G is an isomorphism from G to itself. It is* nontrivial *if it is not the identity.*

As an example, consider the square graph in Figure 1.5. One possible automorphism maps 1 to 2, 2 to 3, 3 to 4 and 4 to 1. The mapping on arcs is determined once the mapping on nodes is given, since there are no multiple arcs. In this case the automorphism corresponds to the geometrical rotation of the square through 90° clockwise, though in general an automorphism does not necessarily have a geometrical interpretation.

The square graph has other automorphisms. For instance, there is one that maps 1 to 3, 2 to 2, 3 to 1 and 4 to 4, corresponding to reflecting the square across the diagonal from 2 to 4. Suppose we wish to count the number of automorphisms of a graph. We could just list all possibilities, but this could be tedious and we might miss some. A good system is to go through the nodes 1 to n in turn, seeing how many possibilities there are for where node i can go, given that nodes 1 to $i-1$ are already fixed. We then multiply together the numbers of possibilities.

Let us follow this system for the square: To start with, 1 can map to any of the four nodes. Given that 1 is fixed, 2 can map to only two of the remaining three nodes, namely the nodes which are adjacent to where 1 has been assigned. Once both 1 and 2 are fixed, the positions of nodes 3 and 4 are determined. There are therefore $4 \times 2 = 8$ possible automorphisms. One of these is of course the identity.

1.2.2 Planar Graphs

In our diagrams we are perfectly happy to allow arcs to cross. For instance the "cube" graph of Figure 1.6 is fine. For some applications it is better to have graphs whose arcs do not cross—for instance if the graph represents a circuit which is to be engraved on a microchip. Say that a graph is *planar* if it can be embedded in the plane (by rearranging the nodes and arcs).

Exercise 1.2.3 *Show that the cube graph of Figure 1.6 is planar, by redrawing the arcs so that they do not cross.*